CSC 1024

PROGRAMMING PRINCIPLES

TITLE OF REPORT:

A PYTHON-BASED INVENTORY MANAGEMENT SYSTEM

Prepared by:

Houda Mohamed	19123900
Tess Njuguna	24050668
Natsai Chidavaenzi	24037913

Prepared for:

ASSOC. PROF. DR ANWAR P.P. ABDUL MAJEED

Date of Submission:

29 DECEMBER 2024

TABLE OF CONTENTS

1.0 Introduction 3
1.1 Load Data Function 3
1.2 Save Data Function 4
1.3 Unique ID Check Function 5
1.4 Add Supplier Function 5
1.5 Place Order Function 6
1.6 Record Sales Function 7
1.7 Generate Reports Function 8
1.8 Add Product Function 10
1.9 Streamlit Interface (User Interface) 11
2.0 Discussion of Implementation Challenges and Solutions 12
3.0 Analysis of the system's strengths and limitations 14
3.1 Strengths 14
3.1.1 Simplicity and Ease of Use 14
3.1.2 Robustness 15
3.1.3 Validation and Error Handling 15
3.1.4 Customizable Reports 15
3.1.5 File Persistence 16
3.2 Limitations 16
3.2.1 Scalability Issues 16
3.2.2 Manual Management of Inventory Adjustments 16
3.2.3 Lack of User Authentication and Authorization 16
4.0 Suggestions for future improvements and enhancements 17
4.1 Improve Reporting 17
4.2 Enhance Security and User Access 17
4.3 Switch to a Database for Scalability 18

Click here to see the video presentation **|** ***Click here for program run-through***

https://anaconda.cloud/share/notebooks/cd9f92d5-5db4-4df5-8f7d-b12a2323f118/overview (Code link- Anaconda)

1.0 Introduction

This is the first section of the program code. There are different types of data files used throughout the program:

PRODUCTS_FILE, the file path that stores product information under *'product.txt.'*

SUPPLIERS_FILE, the file path that stores all supplier information under *'suppliers.txt.'*

ORDERS_FILE, the file path that stores order information from customers under *'orders.txt.'*

SALES_FILE, a file path that stores sales information under *'sales.txt'* helping to keep track of sales transactions.

By defining these file paths as variables, the code enhances maintainability and readability, allowing for easy updates to file names or locations if needed.

Initialize Files Function

The initialize_files function ensures the data files encompassed are available at the start of the application, preventing runtime errors and allows the system to function properly.

Core Functions

1.1 Load Data Function

The load_data function reads data from a specified file, located on a particular file path, and returns it in a structured format.

This function has a try except block to handle exceptions and errors. The block has instructions to open a particular file in read mode ‘r’ and a with statement that automatically ensures the file is closed properly after use.

file.readlines() method reads all the lines of the opened file into a list. The lists of lines are then processed by strip () method that clears any trailing whitespaces and split (‘,’) breaks the string into smaller parts named substrings, based on a specified separator which in this case is the comma.

If the file is non-existent, the FileNotFoundError in the block is issued and is handled by the except part which returns [], and the program can proceed without crashing.

Otherwise, the function returns lines of the file in the form of a list of lists.

1.2 Save Data Function

The data loaded by the previous function is written into a file using this save_data function. This function specifies a particular file path where the data will be saved; this data is the list of lists containing the information from the previous file.

This function opens the file from the specified file path in write mode ‘w,’ a method that will create a new file if it does not exist or overwrite the existing one and uses the with statement to automatically ensure that the file is properly closed after use.

Using a for loop, each item is concatenated by the join () method into a single string, with each element separated by a comma, specified by the (‘,’) method.

Afterwards, the (‘\n’) ensures that each of them is started on a new line in the file and finally written in the file using the file.write () method.

As the purpose of this function is to save data, there is no return value.

1.3 Unique ID Check Function

The is_unique_id function is used to ensure the uniqueness of a given ID within the program. The id_to_check parameter is a formal parameter where the ID that requires verification is entered. Afterwards, the existing_data parameter, a list of current entries, is checked against the ID entered. The purpose of this verification is to avoid duplication and maintain overall data integrity.

The function then extracts the first element (item [0]) from each entry in the existing_data parameter, which is assumed to be the ID. This results in a list of all existing IDs in the system that can be checked against.

Therefore, using the Boolean data type, the program returns True if the ID entered is unique and False if the ID is a duplicate of an existing entry.

1.4 Add Supplier Function

The add_supplier function is designed to facilitate the addition of suppliers and utilizes the Streamlit library, an open-source user interface, to create a user-friendly web interface for interaction.

The function begins with a subheading **"Add Supplier"** that indicates the purpose of the proceeding section. St.text_input parameter allows for user input of the supplier ID, name and supplier contact information. The function employs the ID check from the previous section of code to check the supplier_id against existing ones using existing_ids = [supplier [0] for supplier in suppliers]. If the ID is a duplicate, st.error error message is displayed, and the user is prompted to try a different supplier ID before further processing. If the ID is found to be unique, the user can click the Add Supplier button, and their information is appended to the suppliers list and the

save_data function is called to the information to the relevant list. "Supplier added successfully!" by the st.success is displayed as confirmation.

1.5 Place Order Function

The place_order function is designed to facilitate the placement of orders within the Inventory Management System. This function utilizes Streamlit library to create an interactive interface that allows users to select products and suppliers, specify quantities, and place orders.

The function begins by displaying a subheader, **"Place Supplier Order",** to indicate the purpose of the proceeding section. The function checks if there are any suppliers available by evaluating the suppliers list and if the list is empty, an error message "No suppliers available. Please add suppliers first." is displayed using st.error, prompting the user to add the supplier(s) first. Processing does not continue until the supplier is added. Similarly, the function checks if there are any products available by evaluating the products list. If the list is empty, an error message is displayed,"No products available. Please add products first.", and the user cannot place an order without available products.

The function creates lists of product names and supplier names using list comprehensions: product_choices = [product [1] for product in products]supplier_choices = [supplier [1] for supplier in suppliers]. Using st.selectbox , the user selects a product and supplier from the list of available options. The user is then prompted to enter the number of the product to be ordered using st.number_input and the minimum value is set to 1, ensuring that only positive numbers can be entered.

Next, when the "Place Order" button is clicked, the function retrieves the selected product and supplier from the lists using :product = next((prod for prod in products if prod[1] ==

selected_product), None)supplier = next((supp for supp in suppliers if supp[1] == selected_supplier), None).

If both the product and supplier are found, a unique order ID is generated based on the current length of the orders list: order_id = f"O{len(orders) + 1}" and the current date and time is recorded using datetime.now().strftime("%Y-%m-%d %H:%M:%S"). This information is added to the order.

Next, the function updates the inventory by increasing the quantity of the ordered product [4] = str(int (product [4]) + quantity). Here, it is assumed that the product quantity is stored at index 4 of the product list. The updated quantity is then saved to the products file using save_data(PRODUCTS_FILE, products).

The order details: order ID, product ID, quantity, order date and time and supplier ID, are appended to the orders list. The updated orders list is then saved to the specified file (ORDERS_FILE) using save_data.

If successful, a message is displayed containing the quantity and supplier, "Order placed successfully! {quantity} units of {selected_product} ordered from {selected_supplier}." using st.success, as confirmation.

1.6 Record Sales Function

The record_sales function is designed to facilitate the recording of sales transactions within the Inventory Management System and utilizes the Streamlit library to create an interactive web interface, allowing selection of products, specification of quantities sold and efficient recording of sales. The function begins with a subheader, **"Record Sales,"** indicating the purpose of the proceeding section. The function then checks if there are any products available by evaluating the products list. If the list is empty, an error message is displayed using st.error, prompting the user to add products first before further processing can be done. A list of products is then

displayed : product_choices = [product[1] for product in products], and st.selectbox is used to allow selection of products from the available product listing .The selected product is retrieved from the products list using the expression: product = next((prod for prod in products if prod[1] == selected_product), None). Once retrieved, the function uses st.number_input to create a number input field where the user can enter the quantity sold. The default minimum value is set to 1, ensuring that only positive quantities can be entered. Two date input fields are created using st.date_input for the user to specify the start and end dates for the sales record. The start date defaults to January 1, 2024, and the end date defaults to the current date. When the user clicks the "Record Sales" button, the function first checks the current stock of the selected product: current_stock = int (product [4]) If the quantity sold exceeds the current stock, an error message is displayed.

If the sale is valid, the function updates the product's stock by subtracting the quantity sold from the current stock: product [4] = str(current_stock - quantity_sold) The updated product list is then saved back to the products file using save_data(PRODUCTS_FILE, products).

The function generates a unique sales ID based on the current length of the sales list: sales_id = f"S{len(sales) + 1}". It appends the sales details, including the sales ID, product ID, product name, quantity sold, start and end date, to the sales list. The updated sales list is then saved to the specified file (SALES_FILE) using save_data. Finally, a success message is displayed using st.success, confirming that the sales transaction has been recorded successfully along with the details of the sale, including the remaining stock.

1.7 Generate Reports Function

The generate_reports function is designed to provide users with various reports related to inventory, sales, and supplier orders using the Streamlit library interface. Beginning with the

subheader, **"Reports"**, the function has a dropdown menu using st.selectbox where one can choose the type of report they wish to generate from the available options which are Low Stock, Product Sales and Supplier Orders.

If the user selects "Low Stock", they are prompted to enter a stock threshold using st.number_input. The minimum value is set to 1, and the default value is set to 5. The function then filters the products list, identifying items with stock levels below the specified threshold using low_stock_items = [product for product in products if int (product [4]) < threshold] and thereafter displays a message indicating the products with stock below the threshold. If there are low stock items, a DataFrame is created using pd.DataFrame, and the relevant product details: ID, Name, Description, Price, and Stock are displayed in a table format using st.table. If no products meet the low stock criteria, an informational message is displayed using st.info: "No products with low stock."

If "Product Sales" is selected, the function displays a message indicating that a product sales report is being generated: st.write("Product Sales Report:"). It then checks if there are any recorded sales. If the sales list is not empty, a DataFrame is created with the relevant sales details: Sales ID, Product ID, Product Name, Quantity Sold, Start Date and End Date and displayed in a table format. If no sales have been recorded, an informational message is displayed: st.info ("No sales recorded yet.")

If the user selects "Supplier Orders", the function displays a message indicating that supplier orders are being retrieved :("Supplier Orders:") using st.write. It then checks if there are any recorded orders. If the order list is not empty, a DataFrame is created with the order details: Order ID, Product ID, Quantity, Date and Supplier ID and displayed in a table format. If no orders are available, an informational message is displayed: st.info ("No orders available.")

1.8 Add Product Function

The add_product function facilitates the addition of new products to the system. Beginning with the subheader, **"Add Product",** the function creates input fields for the user to enter the product details: Product ID, Product Name, Product Description, Product Price, and Product Stock. This is facilitated by product_id = st.text_input **,** ("Product ID")name = st.text_input("Product Name")**,** description = st.text_input("Product Description") **,** price_input = st.text_input("Product Price")**,** stock_input = st.text_input("Product Stock").

N.B: Subsequent validation and addition of the product can only occur if the "Add Product" button is clicked.

The function checks if the price input is provided and attempts to convert it to a float. If the price is less than or equal to zero, a value error message: "Price must be a positive number.") is raised using st.error, causing processing to halt. Additionally, if the price input is empty, an error message :("Price cannot be empty.") is displayed because the price cannot be empty.

Similarly, the function checks if the stock input is provided and if it is, it attempts to convert the input to an integer. If the stock is negative, an error message is displayed: "Stock must be a non-negative integer.") and the function returns early. If the stock input is empty, an error message is displayed indicating that the stock cannot be empty: "Stock cannot be empty.")

If both the price and stock inputs are valid, the function checks if the provided product ID is unique by calling the is_unique_id function. If the ID is unique, the product details are appended to the products list and the updated list is saved to (PRODUCTS_FILE) using the save_data function. A success message is displayed using st.success to confirm that the product has been added successfully.

If the product ID already exists, an error message is displayed, prompting the user to use a unique ID: ("Product ID already exists. Please use a unique ID.")

1.9 Streamlit Interface (User Interface)

The Streamlit interface serves as the front-end for the Inventory Management System, providing users with an interactive and user-friendly way to manage products, suppliers, sales, and reports. This section outlines the main components of the interface and how they function.

Title and Menu: The interface begins with a title, "Inventory Management System" displayed prominently at the top using st.title(). A sidebar menu is created using st.sidebar.selectbox, for navigation among different functionalities of the system. The menu options are as follows:

1. Add Product
2. Update Product
3. Record Sales
4. View Inventory
5. Generate Reports
6. Place Supplier Order
7. Add Supplier

If the user selects "Add Product," the interface displays input fields for the user to enter the product ID, name, description, price, and stock. When clicked, the "Add Product" button checks for ID uniqueness using the is_unique_id function. If no duplication occurs, the product is added to the products list, saved to the file and a success message is displayed whereas if the ID is a duplicate, an error message is shown as explained in section 1.8.

If "Update Product" is selected, the interface prompts the user to enter the product ID they wish to update. If the product ID entered is valid, the current details of the product are displayed in

editable fields. The user can modify the name, description, price, and stock. Upon clicking the "Update Product" button, the changes are saved, and a success message is displayed. If the product ID is not found, an error message is shown as discussed in section 1.8.

If the user selects "Record Sales", the record_sales() function is called, which handles the sales recording process as described in section 1.6.

If the user selects "View Inventory," the interface checks if there are any products available. If the products list is empty, an informational message is displayed. Otherwise, a DataFrame is created using pd.DataFrame, and the inventory details: ID, Name, Description, Price, Stock are displayed in a table format using st.table.

If the user selects "Generate Reports," the generate_reports() function is called, allowing users to generate various reports related to inventory, sales, and supplier orders as described in section 1.7.

If the user selects "Place Supplier Order," the place_order() function is called, enabling users to place orders with suppliers as described in section 1.5.

If the user selects "Add Supplier", the add_supplier() function is called, allowing users to add new suppliers as described in section 1.4.

2.0 Discussion of Implementation Challenges and Solutions

Below are the key challenges encountered during the development process, along with the solutions that were implemented to address them.

2.1 Input Validation

Challenge: Ensuring that user inputs, such as product IDs, supplier IDs, and quantities, are valid and conform to expected formats. Invalid inputs could lead to errors or data integrity issues.

Solution: Input validation was implemented using functions like is_unique_id to check for unique IDs and conditional checks to ensure that quantities are positive integers. This prevents users from entering invalid data and maintains the integrity of the system.

2.2 Handling Duplicates

Challenge: Preventing the addition of duplicate products or suppliers, which could lead to confusion and data integrity issues.

Solution: The system checks for existing IDs before adding new entries. For example, in the add_supplier and add_product functions, the program verifies that the entered ID is unique before appending it to the respective lists.

2.3 Error Handling

Challenge: Managing potential errors, such as file errors not found or invalid user inputs, without crashing the program. Unhandled exceptions could lead to poor user experience.

Solution: Try-except blocks were used in critical areas, such as file operations and user input handling, to catch exceptions and provide meaningful error messages. This allows the program to handle errors gracefully and continue running.

2.4 Report Generation

Challenge: The sales report was not fetching data properly, and this caused it to display no sales or outdated information.

Solution: The `load_data` function was introduced and refined to correctly read data from the text files (`products.txt`, `suppliers.txt`, `sales.txt`, and `orders.txt`). Each file was read line-by-

line, and the values were split by commas to ensure that the data was structured properly in the form of lists. This function handles potential `FileNotFoundError` exceptions, returning an empty list if the file does not exist, ensuring the system remains functional even if the data files were missing or not yet created.

2.6 Date and Time Management

Challenge: Accurately capturing and formatting date and time for orders and sales records. Inconsistent date formats lead to confusion and errors in reporting.

Solution: The datetime module was utilized to capture the current date and time in a standardized format. This ensures consistency in how dates are recorded and displayed across the application.

In summary, the development of the Inventory Management System involved addressing various challenges related to input validation, data persistence, user experience and error handling. The solutions implemented not only resolved these challenges but also laid a solid foundation for future enhancements and scalability, ensuring that the system remains robust and user-friendly as it evolves. Continuous testing and user feedback will be essential in maintaining the system's effectiveness and addressing any new challenges that may arise as user needs change and technology advances.

3.0 Analysis of the system's strengths and limitations

3.1 Strengths

3.1.1 Simplicity and Ease of Use

The system has a clear and organized structure, providing dedicated functionalities for managing products, suppliers, orders, and sales. It separates operations into different tabs, such as adding products, placing orders, and generating reports, which enhances maintainability and user experience. The use of Streamlit also ensures a user-friendly interface, which simplifies the user experience. It makes it easy to interact with the system by providing dropdown menus, input fields, and buttons for user interactions. This makes it easier for users to navigate and manage inventory tasks without having to write custom command-line interfaces or manipulate raw data files directly.

3.1.2 Robustness

The system covers important functionalities such as product and supplier management, order placement, sales recording, and inventory tracking. Key business operations (e.g., adding suppliers, placing orders, managing sales) are integrated efficiently, allowing users to run essential operations with minimal effort. The inventory is updated in real-time, allowing users to view current stock levels, place orders, and record sales with up-to-date information. Sales and orders are reflected immediately, giving a real-time view of the system's operations.

3.1.3 Validation and Error Handling

The system validates input data to ensure that only valid entries are processed (e.g., positive integers for quantities and stock, valid price values). This helps prevent incorrect data from being added, which can cause issues when managing inventory or generating reports. It checks for unique IDs (e.g., product IDs and supplier IDs) before adding new entries, which helps maintain data consistency and prevents duplicates.

3.1.4 Customizable Reports

The reporting system is flexible, allowing the user to choose between "Low Stock", "Product Sales", and "Supplier Orders" reports. Users can monitor inventory levels and sales performance effectively, which helps in decision-making.

3.1.5 File Persistence

The system saves and loads data persistently using text files (products.txt, suppliers.txt, orders.txt, sales.txt). This approach ensures that data is retained even if the application is closed, making it a practical solution for inventory management.

3.2 Limitations

3.2.1 Scalability Issues

Using text files for storing data introduces some scalability issues. As the system grows and the amount of data increases, managing and searching through text files can become slower and more cumbersome. It may be more efficient to use a database system (e.g., SQLite, MySQL, PostgreSQL) for better performance and scalability.

3.2.2 Manual Management of Inventory Adjustments

Inventory adjustments are handled manually through the interface (e.g., updating stock levels, placing orders, recording sales). While this is straightforward, it may be error-prone, especially if users frequently forget to update inventory levels or incorrectly enter quantities.

3.2.3 Lack of User Authentication and Authorization

The system lacks any form of user authentication or role-based access control. This means that anyone with access to the interface can perform any action (e.g., add suppliers, update products, and view sales reports).

4.0 Suggestions for future improvements and enhancements

4.1 Improve Reporting

The system currently allows basic reporting like low stock, product sales, and supplier orders. While this is useful, the system could benefit from more detailed and insightful reports that could aid decision-making such as providing reports that show sales over a specific period (e.g., daily, weekly, monthly) to help identify products with high sales and those that are underperforming as well as calculating the revenue generated from each product by multiplying the quantity sold by the price to help identify the most profitable products. The reporting system could additionally benefit from a stock movement analysis functionality which reports on stock levels at different times, including how much stock has been ordered, sold, or returned for the purpose of tracking product turnover and planning future stock orders. Furthermore, the incorporation of charts and graphs (using libraries like matplotlib or plotly) for more visually appealing and insightful reporting will be beneficial. For example, a bar chart showing sales by product category or a line graph displaying stock levels over time.

4.2 Enhance Security and User Access

Security can be enhanced through Implementing a login system to ensure only authorized users can access and perform certain tasks. For example, users can enter a username and password, and depending on their role (e.g., admin, sales staff, warehouse staff), they could be granted

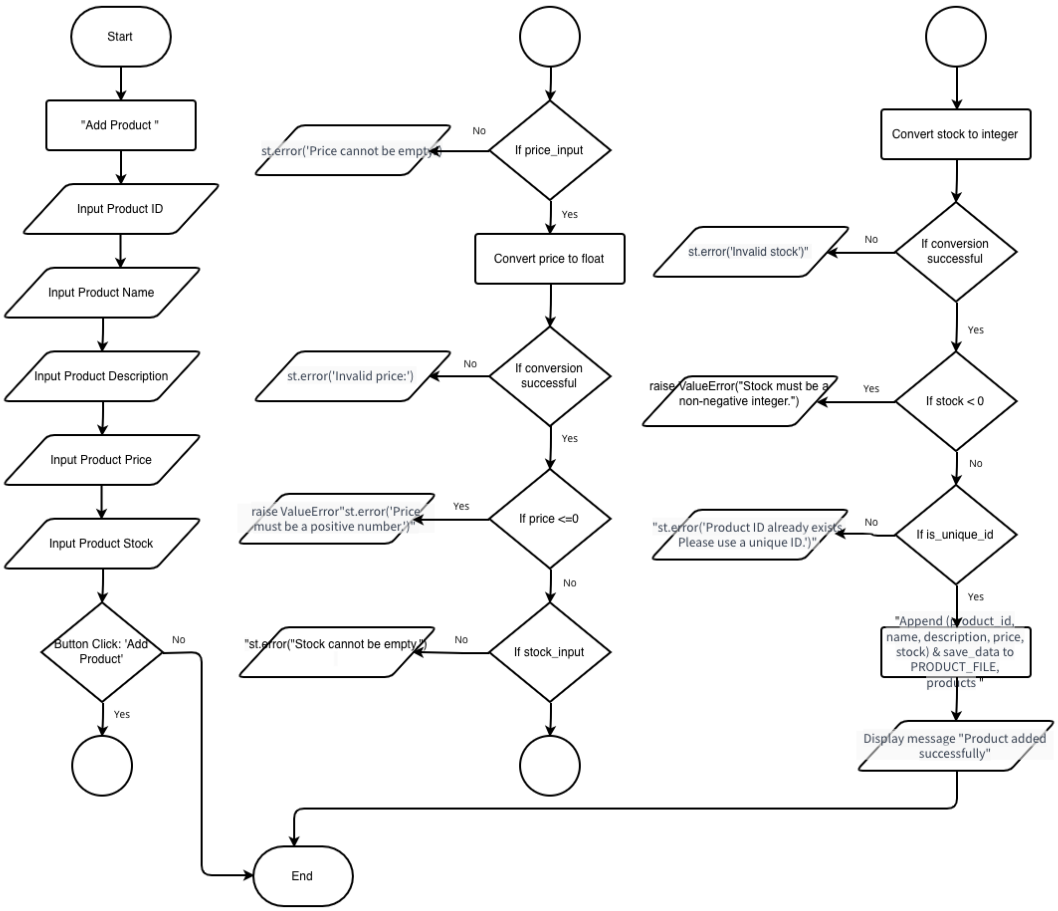
specific permissions. This prevents unauthorized access to sensitive data, such as financial reports or stock levels. This can be done Python libraries such as **streamlit-authenticator** or **flask-login**.These libraries can be integrated to manage user sessions and permissions.

4.3 Switch to a Database for Scalability

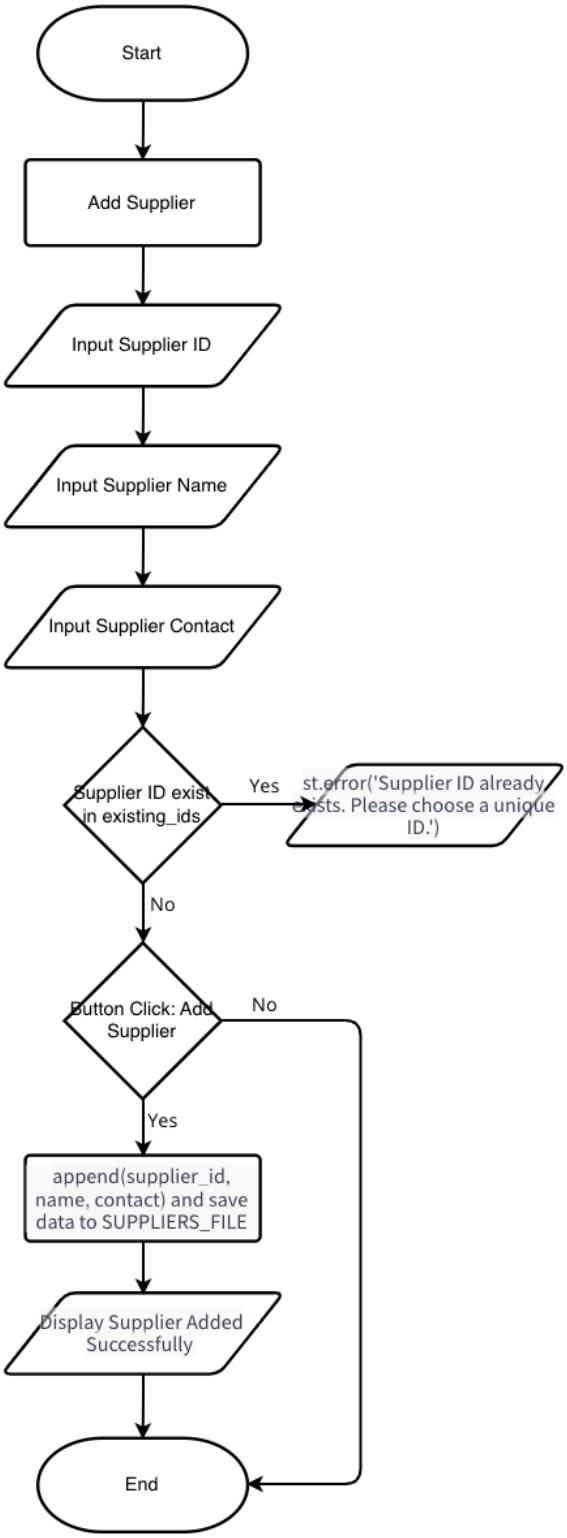
As the system grows and the amount of data increases (more products, sales, orders, and suppliers), managing data via text files will become cumbersome and inefficient. A relational database such as **SQLite**, **PostgreSQL**, or **MySQL** is better suited for handling large datasets.

5.0 FLOWCHARTS

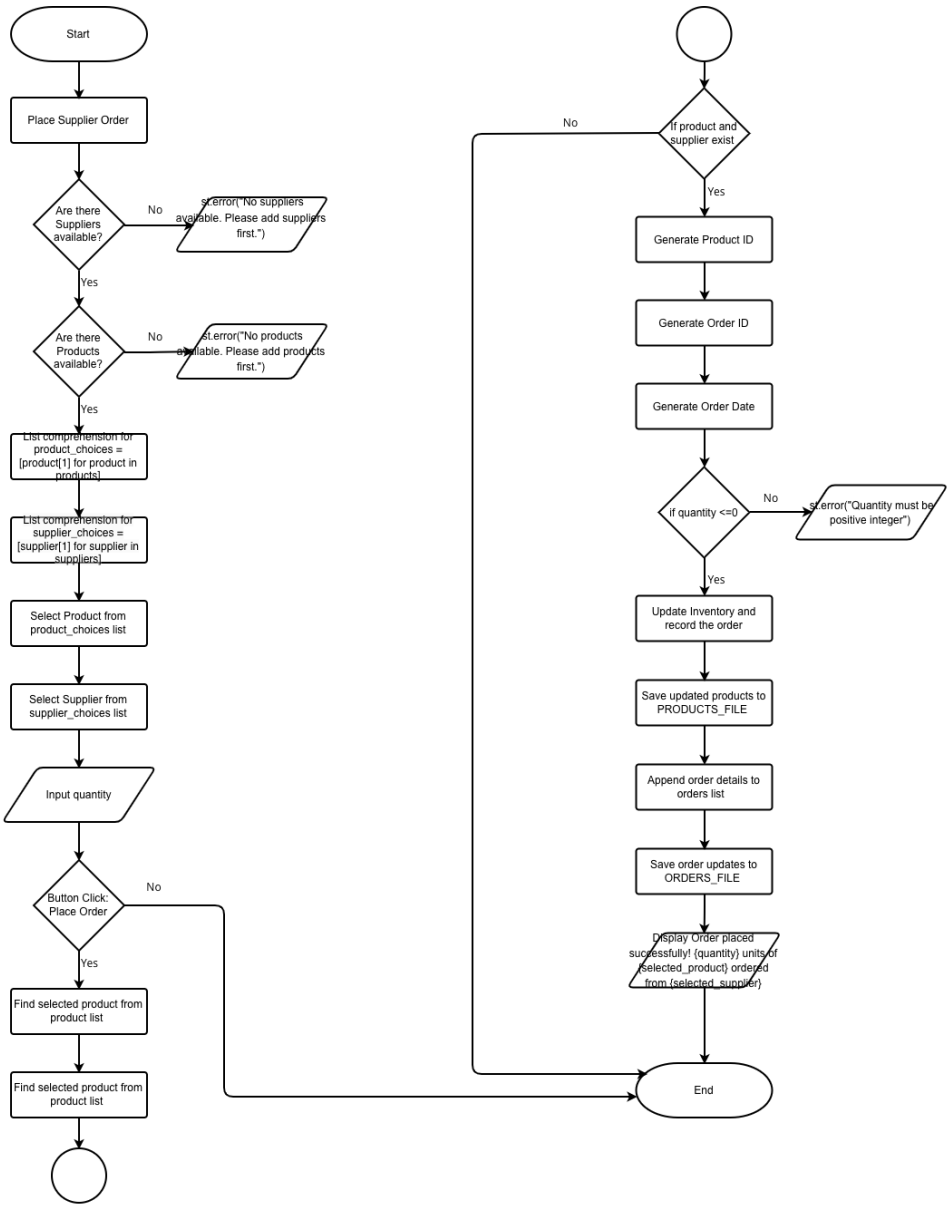
Def add_product ()



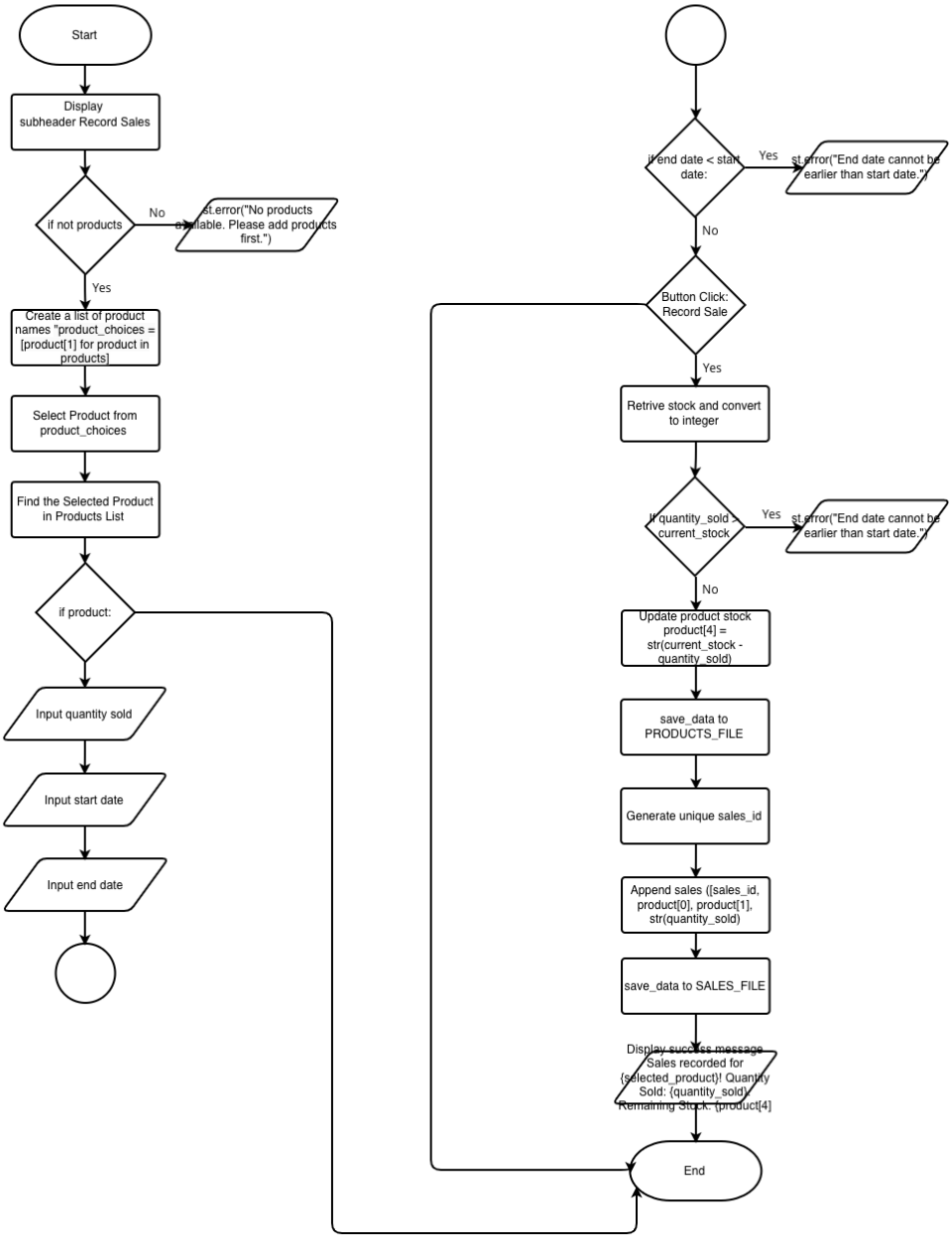
Def add_supplier ():



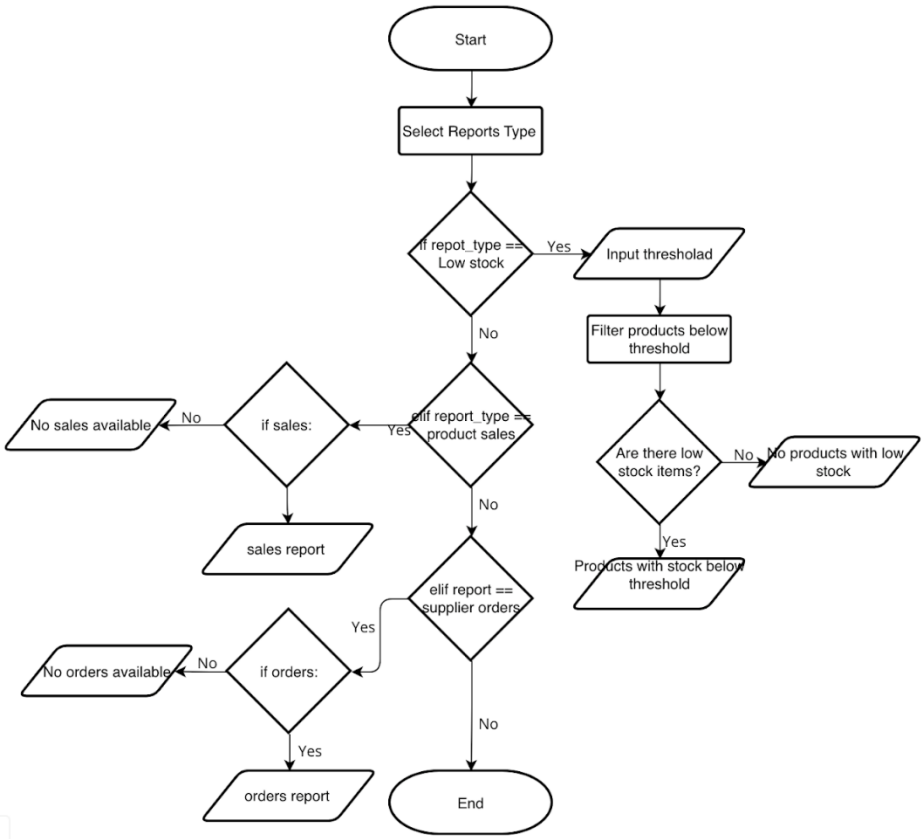
Def place_order():



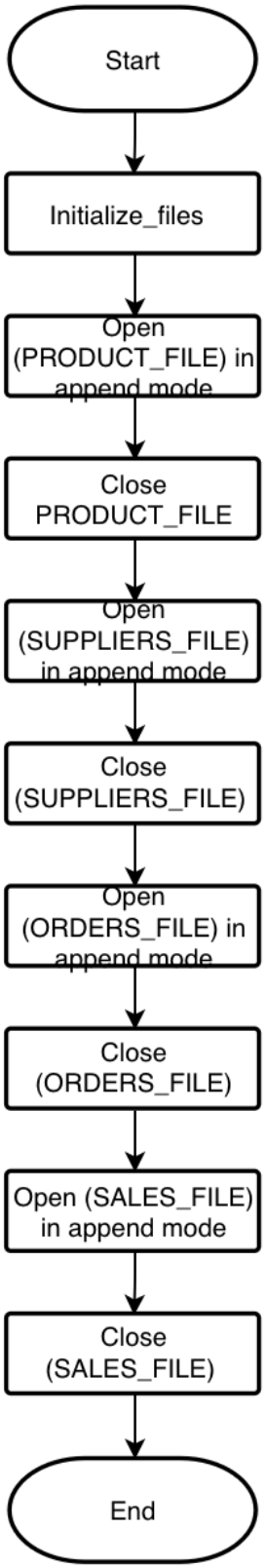
Def sales_order():



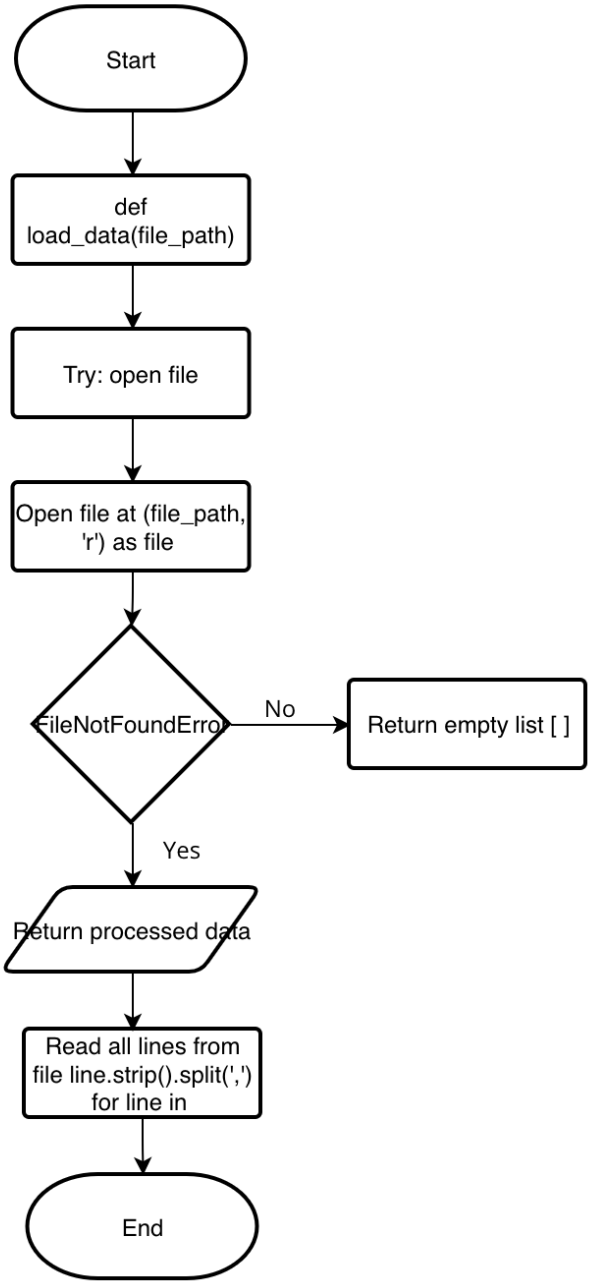
Def generate_reports()



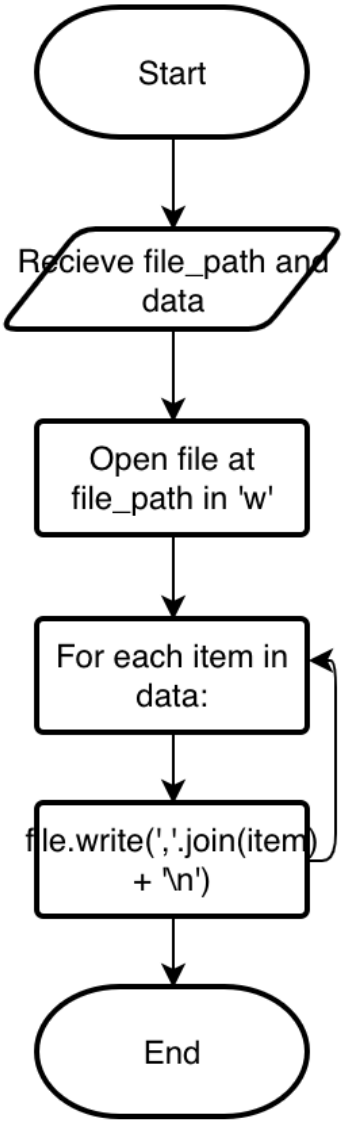
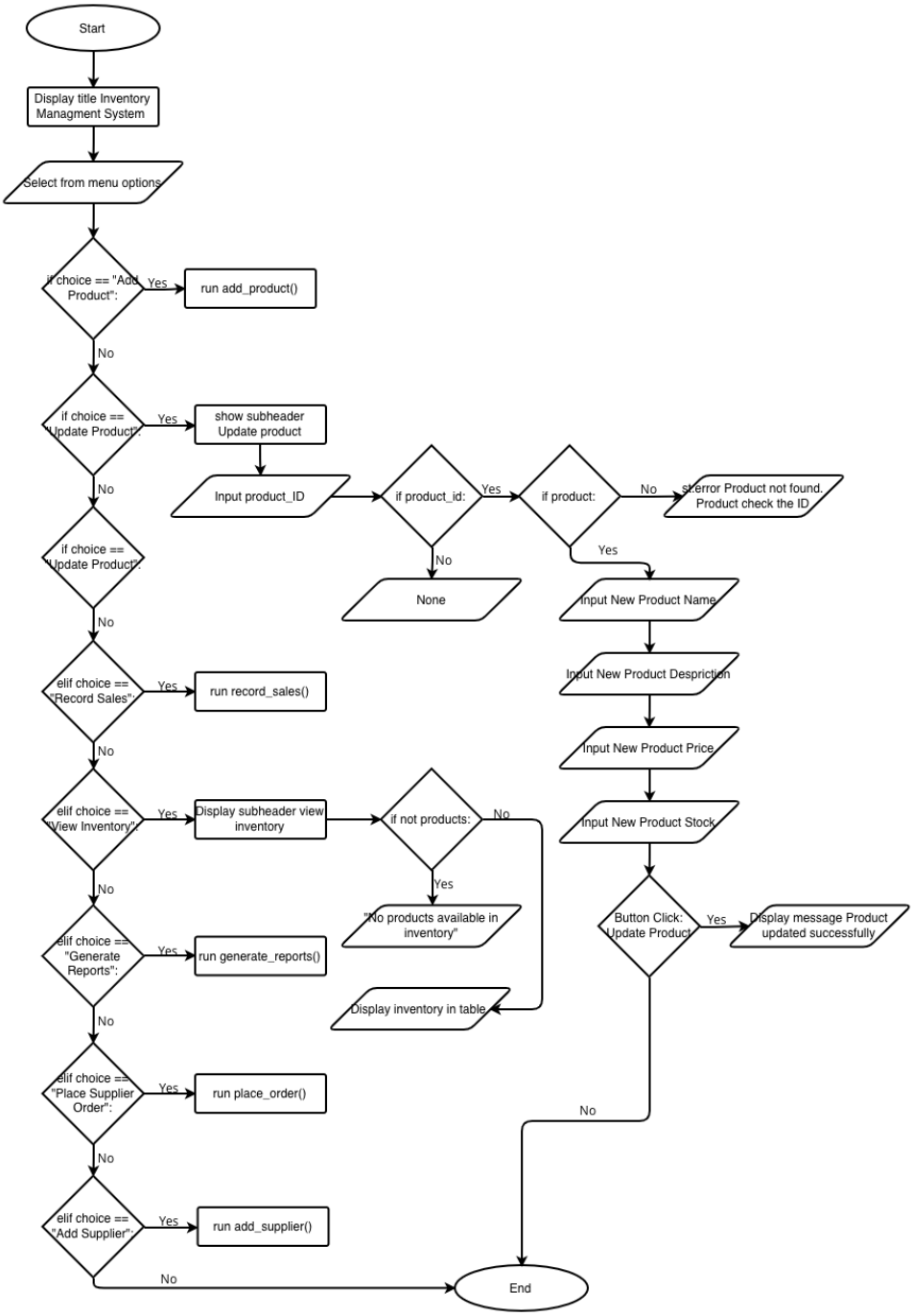
Def initialize_files():



Def load_data(file_path):



def save_data(file_path, data):



Front-End Function (User Interface)